

SQL 활용

강사: 정미나

SQL 분류

- DDL (Data Definition Language)
 - 데이터베이스 구조 정의
- DML (Data Manipulation Language)
 - 데이터를 조작 (삽입, 수정, 삭제)
- DCL (Data Control Language)
 - 데이터 접근 권한 제어
- TCL (Transaction Control Language)
 - 트랜잭션 관리 (Commit, Rollback)

DDL

- 데이터베이스 객체를 정의하거나 변경하는 데 사용
- 주요 명령어
 - CREATE: 테이블, 뷰, 인덱스 생성
 - ALTER: 기존 객체 수정
 - DROP: 객체 삭제
 - TRUNCATE: 테이블 데이터 초기화

DDL - CREATE

- 테이블, 뷰, 인덱스 등 데이터베이스 객체를 생성

```
CREATE TABLE CUSTOMERS (  
    CUSTOMER_ID NUMBER PRIMARY KEY,  
    NAME VARCHAR2(50) NOT NULL,  
    EMAIL VARCHAR2(100) UNIQUE,  
    PHONE VARCHAR2(15),  
    CREATED_AT DATE DEFAULT SYSDATE  
);
```

DDL - ALTER

- 컬럼 추가

```
ALTER TABLE CUSTOMERS ADD ADDRESS VARCHAR2(200);
```

- 컬럼 사이즈 or 데이터 타입 변경

```
ALTER TABLE CLIENTS MODIFY CONTACT_EMAIL VARCHAR2(200);
```

- 컬럼 삭제

```
ALTER TABLE CUSTOMERS DROP COLUMN PHONE;
```

DDL - ALTER

- 컬럼명 변경

```
ALTER TABLE CLIENTS RENAME COLUMN NAME TO FULL_NAME;
```

- 테이블명 변경

```
ALTER TABLE CUSTOMERS RENAME TO CLIENTS;
```

DDL – DROP, TRUNCATE

- 테이블 삭제

```
DROP TABLE CUSTOMERS;
```

- 테이블 데이터 초기화

```
TRUNCATE TABLE CUSTOMERS;
```

DML

- 데이터베이스의 데이터를 조작하는 데 사용
- 주요 명령어
 - SELECT: 데이터 검색
 - INSERT: 데이터 삽입
 - UPDATE: 데이터 수정
 - DELETE: 데이터 삭제

DML – INSERT, SELECT

- 데이터 삽입

```
INSERT INTO CUSTOMERS (CUSTOMER_ID, NAME, EMAIL, PHONE)
VALUES (1, 'JOHN DOE', 'JOHN.DOE@EXAMPLE.COM', '555-1234');
```

- 데이터 조회

```
SELECT NAME, EMAIL
FROM CUSTOMERS
WHERE PHONE = '555-1234';
```

DML – UPDATE, DELETE

- 데이터 변경

```
UPDATE CUSTOMERS  
SET EMAIL = 'JOHN.NEW@EXAMPLE.COM'  
WHERE CUSTOMER_ID = 1;
```

- 데이터 삭제

```
DELETE FROM CUSTOMERS  
WHERE CUSTOMER_ID = 1;
```

DCL

- 데이터베이스 접근 권한을 관리
- 주요 명령어
 - GRANT: 권한 부여
 - REVOKE: 권한 회수

DCL – GRANT, REVOKE

- 권한 부여

```
GRANT SELECT ON CUSTOMERS TO USER1;
```

- 권한 회수

```
REVOKE SELECT ON CUSTOMERS FROM USER1;
```

TCL

- 트랜잭션 관리를 위한 명령어 제공
- 주요 명령어
 - COMMIT: 트랜잭션 확정
 - ROLLBACK: 트랜잭션 취소
 - SAVEPOINT: 특정 지점 저장

TCL – COMMIT, ROLLBACK

- 트랜잭션 확정

```
INSERT INTO CUSTOMERS (CUSTOMER_ID, NAME)
VALUES (2, 'JANE DOE');
COMMIT;
```

- 트랜잭션 취소

```
DELETE FROM CUSTOMERS
WHERE CUSTOMER_ID = 2;
ROLLBACK;
```

SELECT

SELECT

- SQL에서 데이터를 검색하는 명령어
- 테이블에서 원하는 데이터를 추출
- 조건, 정렬, 그룹핑 등을 사용하여 복잡한 질의 가능
- 업무에서 가장 많이 사용하게 될 구문

SELECT

- 기본 문법

SELECT 컬럼명 FROM 테이블명;

- 예시

SELECT * FROM EMPLOYEES;

SELECT NAME, SALARY FROM EMPLOYEES;

ALIAS

- ALIAS(별칭)의 정의
 - SQL 쿼리에서 컬럼이나 테이블에 부여하는 임시 이름
 - 결과 집합의 가독성을 높이고 복잡한 쿼리를 간결하게 작성할 수 있도록 돕는 기능
- ALIAS의 특징
 - 테이블이나 컬럼 이름 대신 사용할 수 있음
 - 데이터베이스 구조에 영향을 주지 않는 임시 이름
 - 컬럼 ALIAS의 경우 AS 키워드를 사용하지만 생략도 가능

컬럼 ALIAS

- 사용 이유
 - 연산 결과에 의미를 부여
 - 출력 결과의 컬럼 이름을 변경하여 가독성을 높임

- 문법

SELECT 컬럼명 (AS) 별칭

FROM 테이블명;

컬럼 ALIAS

- 기본 사용

```
SELECT NAME AS EMP_NM,  
        SALARY AS SAL
```

```
FROM EMPLOYEES;
```

- 연산 결과에 ALIAS 추가

```
SELECT SALARY * 12 AS ANNUAL_SAL  
FROM EMPLOYEES;
```

테이블 ALIAS

- 사용 이유
 - 테이블 이름이 길거나 복잡할 때 간결한 이름으로 대체
 - JOIN 쿼리에서 테이블 이름을 축약하여 사용

- 문법

SELECT 컬럼명 FROM 테이블명 별칭;

테이블 ALIAS

- 단일 테이블 사용

```
SELECT E.NAME, E.SALARY  
FROM EMPLOYEES E;
```

- JOIN 쿼리에서 사용

```
SELECT E.NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E, DEPARTMENTS D  
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

ALIAS 활용

- 조합 사용

```
SELECT E.NAME AS EMP_NAME,  
       D.DEPARTMENT_NAME AS DEPT_NAME  
FROM EMPLOYEES E, DEPARTMENTS D  
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

- 특수문자 또는 공백, 한글 포함

```
SELECT NAME AS "이름",  
       SALARY AS "연간 급여"  
FROM EMPLOYEES;
```

NULL

- 데이터베이스에서 NULL은 "값이 없음"을 의미
- 숫자, 문자 등 어떤 데이터도 저장되지 않은 상태
- 데이터가 "비어 있다"고 간주되며, 이는 **0**이나 빈 문자열("")과는 다름
- 정의되지 않은 상태

NULL 연산

- NULL 값은 다른 값과 비교할 수 없음
- 비교 연산 (=, <>, <, >, <=, >=)을 수행하면 항상 UNKNOWN 반환 -> FALSE로 귀결
- 예시

```
SELECT * FROM EMPLOYEES WHERE MANAGER_ID = NULL;
```

```
-> SELECT * FROM EMPLOYEES WHERE MANAGER_ID IS NULL;
```

WHERE

WHERE

- WHERE 절로 조건을 지정하여 데이터를 필터링
- SELECT, UPDATE, DELETE와 함께 사용
- 예시

```
SELECT NAME FROM EMPLOYEES WHERE SALARY > 5000;
```

```
SELECT * FROM EMPLOYEES WHERE DEPARTMENT_ID = 10;
```

WHERE 절 연산자

- 다양한 연산자를 활용하여 복잡한 조건 설정 가능
- 비교 연산자: =, <>, >, <, >=, <=
- 논리 연산자: AND, OR, NOT
- 특수 연산자: LIKE, IN, BETWEEN, IS NULL

비교 연산자

- =: 값이 같은지 비교
- <>: 값이 다른지 비교
- >, <: 값이 크거나 작는지 비교
- >=, <=: 값이 크거나 같음 또는 작거나 같음을 비교
- 예시

```
SELECT * FROM EMPLOYEES WHERE SALARY > 5000;
```

논리 연산자 - AND

- 여러 조건을 모두 만족해야 함
- 조건이 TRUE인 경우만 결과 반환
- 예시

```
SELECT * FROM EMPLOYEES
```

```
WHERE DEPARTMENT_ID = 10 AND SALARY > 5000;
```

논리 연산자 - OR

- 여러 조건 중 하나만 만족하면 됨
- 조건이 TRUE인 경우 결과 반환
- 예시

```
SELECT * FROM EMPLOYEES
```

```
WHERE DEPARTMENT_ID = 10 OR DEPARTMENT_ID = 20;
```

논리 연산자 - NOT

- 조건의 결과를 반전시킴
- TRUE는 FALSE로, FALSE는 TRUE로 반환
- 예시

```
SELECT * FROM EMPLOYEES
```

```
WHERE NOT SALARY > 5000;
```

특수 연산자 - LIKE

- 특정 패턴을 가진 데이터를 검색
- %: 0개 이상의 문자를 대체
- _: 1개의 문자를 대체
- 예시

```
SELECT * FROM CUSTOMERS  
WHERE NAME LIKE 'C%';
```

특수 연산자 - LIKE

패턴	설명	예시	일치 결과
'A%'	'A'로 시작하는 모든 문자열	LIKE 'A%'	Apple, April, Alone
'%z'	'z'로 끝나는 모든 문자열	LIKE '%z'	Buzz, Jazz
'%cat%'	'cat'이 포함된 모든 문자열	LIKE '%cat%'	Bobcat, Wildcat lake
'J_n%'	'J'로 시작하고, 세 번째 문자가 'n'인 문자열	LIKE 'J_n%'	Jane, Jonson
'_o_e'	두 번째가 'o', 네 번째가 'e'인 4글자 문자열	LIKE '_o_e'	Rose, Move
'%'	모든 문자열	LIKE '%'	Apple, Buzz, Cat, ...
'_'	정확히 1개의 임의의 문자	LIKE '_'	A, B, C (단일 문자)
'50%'	'50'로 시작하는 모든 문자열	LIKE '50%'	500, 501, 50% Discount
'CW%A%' ESCAPE 'W'	'C%A'로 시작하는 모든 문자열	LIKE 'C\%A%' ESCAPE '\'	C%A123, C%A_test

특수 연산자 - LIKE

- ESCAPE
- 기본 문법

SELECT 컬럼명

FROM 테이블명

WHERE 컬럼명 LIKE '패턴' ESCAPE '문자';

특수 연산자 - LIKE

- %를 일반 문자로 처리

```
SELECT NAME
```

```
FROM CUSTOMERS
```

```
WHERE NAME LIKE '%rW%%' ESCAPE 'W';
```

➤ 패턴: 'r%'를 포함하는 데이터

특수 연산자 - LIKE

- _를 일반 문자로 처리

```
SELECT NAME
```

```
FROM CUSTOMERS
```

```
WHERE NAME LIKE '%#_3%' ESCAPE '#';
```

➤ 패턴: ' _3'을 포함하는 데이터

특수 연산자 - LIKE

- ESCAPE 주의점
 - ESCAPE로 지정한 문자가 패턴 문자열에 포함되어야 함
 - ESCAPE 없이 와일드카드 문자를 패턴으로 인식하면 예상치 못한 결과가 나올 수 있음

Quiz

파일명[FILE_NAME]이 'Data_2026' 으로 시작하는 데이터를
파일정보[FILE_INFO] 테이블에서 조회하는 SQL을 작성하세요.

특수 연산자 - IN

- 지정된 값 중 하나와 일치하는 데이터를 찾을
- OR 조건으로 대체 가능
- 예시

```
SELECT * FROM EMPLOYEES
```

```
WHERE DEPARTMENT_ID IN (10, 20, 30);
```

특수 연산자 - BETWEEN

- 지정된 범위 내에 있는 값을 찾음
- 범위는 시작 값과 끝 값 포함
- 예시

```
SELECT * FROM EMPLOYEES
```

```
WHERE SALARY BETWEEN 3000 AND 5000;
```

특수 연산자 – IS NULL

- NULL 값을 가진 데이터를 찾음
- NOT을 붙이면 NULL 값이 아닌 데이터를 찾음
- 예시

```
SELECT * FROM CUSTOMERS WHERE EMAIL IS NULL;
```

```
SELECT * FROM CUSTOMERS WHERE EMAIL IS NOT NULL;
```

연산자의 우선순위

- SQL에서 여러 연산자를 조합한 조건문을 사용할 때 어떤 연산자를 먼저 실행할지 결정하는 순서

우선순위	연산자	설명
1	<code>()</code> (괄호)	괄호 내부의 연산을 가장 먼저 수행
2	비교 연산자 (<code>=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>)	값의 비교를 수행
3	논리 연산자 <code>NOT</code>	조건을 반전시킴
4	논리 연산자 <code>AND</code>	모든 조건이 참이어야 참
5	논리 연산자 <code>OR</code>	하나의 조건이라도 참이면 참

연산자의 우선순위

```
SELECT *  
FROM EMPLOYEES  
WHERE SALARY > 5000 OR DEPARTMENT_ID = 10 AND DEPARTMENT_ID = 20;
```

```
SELECT *  
FROM EMPLOYEES  
WHERE (SALARY > 5000 OR DEPARTMENT_ID = 10) AND DEPARTMENT_ID = 20;
```

연산자의 우선순위

```
SELECT * FROM EMPLOYEES  
WHERE NOT SALARY > 5000 AND JOB_ID = 'DEV';
```

```
SELECT * FROM EMPLOYEES  
WHERE NOT (SALARY > 5000 AND JOB_ID = 'DEV');
```

실습

금융 서비스 Database

SQL Developer & 실습 스크립트

실습 배경 설명 - 테이블

- CUSTOMERS: 고객 정보 관리
- ACCOUNTS: 계좌 정보 관리
- TRANSACTIONS: 거래 내역 관리
- BRANCHES: 은행 지점 관리
- LOANS: 대출 정보 관리

실습 배경 설명 - 관계

- CUSTOMERS ↔ ACCOUNTS
 - 1:N 관계 (한 고객이 여러 계좌를 가질 수 있음)
- ACCOUNTS ↔ TRANSACTIONS
 - 1:N 관계 (한 계좌에 여러 거래가 발생)
- CUSTOMERS ↔ LOANS
 - 1:N 관계 (한 고객이 여러 대출을 받을 수 있음)
- BRANCHES ↔ LOANS
 - 1:N 관계 (한 지점에서 여러 대출이 이루어짐)

SQL 작성 1

고객 테이블에서 이름에 "%" 기호가 포함된 고객의 이름과 이메일을 출력하시오.

계좌 테이블에서 계좌 유형이 "SAVINGS" 또는 "CREDIT"이면서 잔액이 30,000 이상 70,000 이하인 계좌의 계좌 ID와 잔액을 출력하시오.

거래 테이블에서 거래 유형이 "DEPOSIT"이고 거래 금액이 1,000 이상인 거래의 거래 ID와 금액을 출력하시오.

대출 테이블에서 대출 금액이 50,000 이상이며 대출 상태가 "REJECTED"가 아닌 대출 ID와 금액을 출력하시오.

ORDER BY

ORDER BY

- SQL에서 데이터를 정렬하는 데 사용되는 절
- 기본적으로 오름차순(ASC) 정렬
- 내림차순 정렬 시 DESC 키워드 사용
- 하나 이상의 컬럼을 기준으로 정렬 가능

ORDER BY

- 문법

```
SELECT 컬럼명 FROM 테이블명  
ORDER BY 컬럼명 [ASC|DESC];
```

- 예시

```
SELECT NAME, SALARY FROM EMPLOYEES  
ORDER BY SALARY DESC;
```

다중 컬럼 정렬

- 여러 컬럼을 기준으로 정렬 가능
- 우선순위는 지정한 컬럼의 순서에 따름
- 예시

```
SELECT NAME, DEPARTMENT_ID, SALARY  
FROM EMPLOYEES  
ORDER BY DEPARTMENT_ID ASC, SALARY DESC;
```

ORDER BY와 NULL

- Oracle에서 NULL 값은 최대값으로 간주
- NULLS FIRST 또는 NULLS LAST로 명시적 설정 가능
- 예시

```
SELECT NAME, SALARY
```

```
FROM EMPLOYEES
```

```
ORDER BY SALARY ASC NULLS FIRST;
```

SQL 작성 2

고객 테이블에서 모든 고객의 이름과 이메일을 이메일 오름차순으로 정렬하되, 이메일이 없는 고객은 맨 앞으로 보내고 이메일이 같거나 없는 경우 이름 오름차순으로 정렬하여 출력하시오.

계좌 테이블에서 잔액이 50,000 이상인 계좌를 대상으로 계좌유형 오름차순 -> 잔액 내림차순으로 정렬하여 출력하시오.

거래 테이블에서 거래 유형을 기준으로 오름차순, 거래 금액을 기준으로 내림차순 정렬하여 거래ID와 거래금액을 출력하시오.

DISTINCT

DISTINCT

- SQL에서 중복된 데이터를 제거하여 고유한 값만 반환하는 키워드
- SELECT 문과 함께 사용하여 특정 컬럼의 고유한 값만 조회할 수 있음
- 데이터의 요약/집계에 유용
- DISTINCT 다음에 여러 개의 컬럼이 나올 경우 모두 중복의 기준에 적용됨

DISTINCT

- 기본 문법

SELECT DISTINCT 컬럼명

FROM 테이블명

WHERE 조건;

- 예시: 계좌 테이블에서 계좌 타입을 중복 없이 조회

SELECT DISTINCT ACCOUNT_TYPE

FROM ACCOUNTS;

DISTINCT 예시

- 거래 테이블에서 거래 유형을 중복 없이 조회

```
SELECT DISTINCT TRANSACTION_TYPE  
FROM TRANSACTIONS;
```

DISTINCT 예시

- 계좌 테이블에서 고객 ID, 계좌 타입 셋을 중복 없이 조회

```
SELECT DISTINCT CUSTOMER_ID, ACCOUNT_TYPE  
FROM ACCOUNTS;
```

GROUP BY

데이터 그룹핑

- 데이터베이스에서 데이터를 특정 기준으로 묶는 작업
- GROUP BY 절을 사용하여 데이터를 그룹핑
- 집계 함수와 함께 사용하여 그룹별 요약 정보 제공

GROUP BY

- SELECT 문에서 데이터를 그룹으로 묶는 절
- 그룹별로 집계 결과를 반환
- WHERE 절과 함께 사용 가능

GROUP BY

- 문법

```
SELECT 컬럼, 집계함수 FROM 테이블  
GROUP BY 컬럼;
```

- 예시

```
SELECT DEPARTMENT_ID, AVG(SALARY)  
FROM EMPLOYEES  
GROUP BY DEPARTMENT_ID;
```

GROUP BY와 집계 함수

- 집계 함수는 그룹별로 데이터 요약 정보를 제공
- 주요 집계 함수
 - COUNT(): 행의 개수 반환
 - SUM(): 합계 반환
 - AVG(): 평균 반환
 - MAX(): 최대값 반환
 - MIN(): 최소값 반환

GROUP BY 예제

- 부서별 평균 급여 계산

```
SELECT DEPARTMENT_ID, AVG(SALARY) AS AVG_SALARY  
FROM EMPLOYEES  
GROUP BY DEPARTMENT_ID;
```

GROUP BY 예제

- 직책별 최대 급여와 최소 급여 계산

```
SELECT JOB_ID,  
       MAX(SALARY) AS MAX_SALARY,  
       MIN(SALARY) AS MIN_SALARY  
FROM EMPLOYEES  
GROUP BY JOB_ID;
```

GROUP BY와 WHERE

- WHERE 절은 GROUP BY 전에 조건을 필터링
- 예시: 5,000 이상 급여를 가진 직원만 그룹핑

```
SELECT DEPARTMENT_ID, COUNT(*) AS EMP_COUNT  
FROM EMPLOYEES  
WHERE SALARY >= 5000  
GROUP BY DEPARTMENT_ID;
```

GROUP BY와 HAVING

- HAVING 절은 GROUP BY 이후 그룹별 조건 필터링
- 예시: 평균 급여가 6,000 이상인 부서만 출력

```
SELECT DEPARTMENT_ID, AVG(SALARY) AS AVG_SALARY  
FROM EMPLOYEES  
GROUP BY DEPARTMENT_ID  
HAVING AVG(SALARY) >= 6000;
```

GROUP BY 실습 예제

- 계좌 유형별 평균 잔액 조회

```
SELECT ACCOUNT_TYPE, AVG(BALANCE) AS AVG_BALANCE  
FROM ACCOUNTS  
GROUP BY ACCOUNT_TYPE  
ORDER BY AVG_BALANCE DESC;
```

GROUP BY 실습 예제

- 거래 유형별 총 거래 금액 조회

```
SELECT TRANSACTION_TYPE,  
       SUM(AMOUNT) AS TOTAL_AMOUNT  
FROM TRANSACTIONS  
GROUP BY TRANSACTION_TYPE;
```

GROUP BY 실습 예제

- 고객당 계좌 수 및 평균 잔액을 구하고 평균 잔액이 50,000 이상인 고객만 출력

```
SELECT CUSTOMER_ID,  
        COUNT(*) AS ACCOUNT_COUNT,  
        AVG(BALANCE) AS AVG_BALANCE  
FROM ACCOUNTS  
GROUP BY CUSTOMER_ID  
HAVING AVG(BALANCE) >= 50000;
```

SQL 작성 3

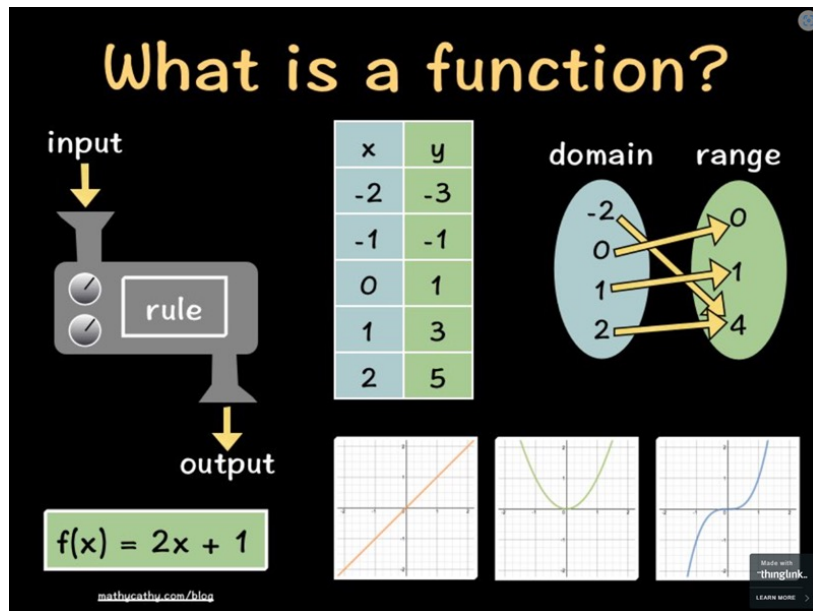
승인된 대출(APPROVED)의 총 대출 금액과 해당 대출 건수를 출력하시오.

각 월별 총 거래 금액을 계산하고, 거래 금액이 10,000 이상인 달만 출력하시오.

각 지점에서 승인된 대출(APPROVED)의 평균 대출 금액과 총 대출 금액을 계산하시오.

함수

함수란



- 데이터를 처리하거나 계산하여 결과를 반환하는 사전 정의된 명령어
- 데이터 검색, 변환, 요약 등을 자동화하여 효율성을 높임
- 단일 값 함수와 집계 함수로 구분

단일 행 함수(Single-Row Functions)

- 한 행에 대해 하나의 결과를 반환
- 문자 함수: 문자열 변환 및 처리
- 숫자 함수: 수학적 계산
- 날짜 함수: 날짜 및 시간 처리
- 변환 함수: 데이터 형식 변환
- NULL 처리 함수: NULL 값 처리

집계 함수(Aggregate Functions)

- 여러 행을 그룹으로 묶어 하나의 결과를 반환
- COUNT(): 행의 개수 반환
- SUM(): 합계 계산
- AVG(): 평균 계산
- MAX(): 최대값 반환
- MIN(): 최소값 반환

문자 함수

- 문자열을 다루기 위한 함수
- 문자열 변환, 검색, 길이 계산 등을 제공
- 주요 함수
 - UPPER(), LOWER()
 - LENGTH(), SUBSTR()
 - INSTR()와 TRIM()
 - LPAD()와 RPAD()
 - REPLACE()와 CONCAT()

문자 함수

- UPPER()와 LOWER(): 대소문자 변환

```
SELECT UPPER('hello') AS UPPERCASE,  
       LOWER('WORLD') AS LOWERCASE  
FROM DUAL;
```

문자 함수

- LENGTH()와 SUBSTR(): 문자열 길이와 일부 추출

```
SELECT LENGTH('SQL Function') AS LENGTH,  
        SUBSTR('SQL Function', 5, 8) AS SUBSTRING  
FROM DUAL;
```

문자 함수

- INSTR()와 TRIM(): 특정 문자열의 처음 위치 출력과 공백 제거

```
SELECT INSTR('Oracle SQL', 'SQL') AS POSITION,  
       TRIM(' Oracle SQL ') AS TRIMMED_STRING  
FROM DUAL;
```

문자 함수

- LPAD()와 RPAD(): 문자열 채움

```
SELECT LPAD('SQL', 10, '*') AS LEFT_PADDED,  
       RPAD('SQL', 10, '*') AS RIGHT_PADDED  
FROM DUAL;
```

문자 함수

- REPLACE()와 CONCAT(): 문자열 대체와 두 문자열 연결
SELECT REPLACE('Hello SQL', 'SQL', 'Oracle') AS REPLACED_STRING,
CONCAT('Hello', ' World') AS CONCATENATED_STRING
FROM DUAL;

숫자 함수

- 숫자를 다루기 위한 함수
- 주요 함수
 - ROUND(), TRUNC()
 - ABS(), MOD()
 - CEIL()와 FLOOR()

숫자 함수

- ROUND()와 TRUNC(): 숫자 반올림과 자르기

```
SELECT ROUND(123.456, 2) AS ROUNDED,  
       TRUNC(123.456, 2) AS TRUNCATED  
FROM DUAL;
```

숫자 함수

- ABS()와 MOD(): 절대값과 나머지 계산

```
SELECT ABS(-123) AS ABSOLUTE,  
       MOD(10, 3) AS REMAINDER  
FROM DUAL;
```

숫자 함수

- CEIL()와 FLOOR(): 숫자를 올림과 내림

```
SELECT CEIL(123.456) AS CEILED,
```

```
       FLOOR(123.456) AS FLOORED
```

```
FROM DUAL;
```

날짜 함수

- 날짜 및 시간 데이터를 다루기 위한 함수
- 주요 함수
 - SYSDATE, CURRENT_DATE
 - ADD_MONTHS(), MONTHS_BETWEEN()
 - NEXT_DAY()와 LAST_DAY()
 - EXTRACT()와 TRUNC()

날짜 함수

- SYSDATE와 CURRENT_DATE: 현재 날짜와 시간 반환
(SYSDATE: DB 서버시간 / CURRENT_DATE: 세션 타임존 기준 시간)

```
SELECT SYSDATE AS SYS_DATE
```

```
      CURRENT_DATE AS SESSION_DATE
```

```
FROM DUAL;
```

날짜 함수

- ADD_MONTHS()와 MONTHS_BETWEEN(): 날짜 계산

```
SELECT ADD_MONTHS(SYSDATE, 2) AS TWO_MONTHS_LATER,  
       MONTHS_BETWEEN(SYSDATE,  
                       TO_DATE('2024-01-01', 'YYYY-MM-DD')) AS MONTH_DIFF  
FROM DUAL;
```

날짜 함수

- NEXT_DAY()와 LAST_DAY(): 지정된 요일의 다음 날짜와 해당 월의 마지막 날짜 반환

```
SELECT NEXT_DAY(SYSDATE, '일요일') AS NEXT_SUNDAY,  
       LAST_DAY(SYSDATE) AS END_OF_MONTH  
FROM DUAL;
```

날짜 함수

- EXTRACT()와 TRUNC(): 날짜에서 특정 요소 추출과 날짜를 특정 단위로 자름

```
SELECT EXTRACT(YEAR FROM SYSDATE) AS CURRENT_YEAR,  
       TRUNC(SYSDATE, 'MM') AS START_OF_MONTH  
FROM DUAL;
```

변환 함수

- 데이터 타입을 다른 타입으로 변환하는 함수
- 주요 함수
 - TO_CHAR()
 - TO_DATE()
 - TO_NUMBER()

변환 함수

- TO_CHAR(), TO_DATE(), TO_NUMBER(): 데이터 형식 변환
SELECT TO_CHAR(SYSDATE, 'YYYY-MM-DD') AS DATE_STRING,
TO_DATE('2024-12-01', 'YYYY-MM-DD') AS DATE_VALUE,
TO_NUMBER('1234') AS NUMERIC_VALUE
FROM DUAL;

NULL 처리 함수

- NULL 값을 처리하는 함수
- 주요 함수
 - NVL(), NVL2()
 - COALESCE()
 - NULLIF()

NULL 처리 함수

- NVL(): NULL 값 대체

```
SELECT NVL(NULL, 'Default Value') AS RESULT  
FROM DUAL;
```

NULL 처리 함수

- NVL2(): NULL 여부에 따라 다른 값을 반환

```
SELECT NVL2(NULL, 'Not Null', 'Null') AS RESULT  
FROM DUAL;
```

NULL 처리 함수

- COALESCE(): NULL이 아닌 첫 번째 값 반환

```
SELECT COALESCE(NULL, NULL, 'First Non-NULL') AS RESULT  
FROM DUAL;
```

NULL 처리 함수

- NULLIF(): 두 값이 같으면 NULL 반환

```
SELECT NULLIF(1, 1) AS RESULT1, NULLIF(1, 2) AS RESULT2  
FROM DUAL;
```

CASE WHEN과 DECODE

CASE WHEN

- SQL에서 조건에 따라 값을 반환하는 문법
- SELECT, UPDATE, DELETE 문 등에서 사용 가능
- 가독성이 뛰어나 복잡한 조건을 처리할 때 유용

CASE WHEN

CASE

WHEN 조건1 THEN 결과1

WHEN 조건2 THEN 결과2

ELSE 결과

END

CASE 컬럼명

WHEN 값 THEN 결과1

WHEN 값 THEN 결과2

ELSE 결과

END

CASE WHEN 예제

```
SELECT EMPLOYEE_ID,  
       CASE WHEN SALARY > 10000 THEN 'HIGH'  
            WHEN SALARY BETWEEN 5000 AND 10000 THEN 'MEDIUM'  
            ELSE 'LOW'  
       END AS SALARY_LEVEL  
FROM EMPLOYEES;
```

DECODE

- 특정 값에 따라 다른 값을 반환하는 함수
- SELECT 문에서 주로 사용
- CASE WHEN보다 간단한 조건 처리에 적합
- 문법

DECODE(표현식, 조건1, 결과1, 조건2, 결과2, ..., 기본값)

DECODE 예제

```
SELECT EMPLOYEE_ID,  
       DECODE(JOB_ID, 'ADMIN', '관리자',  
                'DEV', '개발자',  
                'HR', '인사담당자',  
                '기타') AS DEPARTMENT_NAME  
FROM EMPLOYEES;
```

CASE WHEN vs DECODE

- CASE WHEN
 - 조건 처리에 유연함
 - 여러 조건 및 복잡한 로직 처리 가능
- DECODE
 - 간단한 값 매핑에 적합
 - 특정 열의 값 기반으로 처리

JOIN

JOIN

- 여러 테이블의 데이터를 결합하여 하나의 결과 집합을 생성하는 SQL 절
- 기본적으로 두 개 이상의 테이블을 연결
- 테이블 간 관계를 활용하여 데이터 조회

JOIN의 필요성

- 데이터의 일관성과 중복 방지를 위해 데이터를 여러 테이블로 나누어 저장 (정규화)
- 나누어진 데이터를 조합하여 원하는 정보를 조회할 때 JOIN 사용

JOIN의 종류

- INNER JOIN
- OUTER JOIN
- SELF JOIN
- CROSS JOIN
- NATURAL JOIN

INNER JOIN

- 두 테이블 간에 일치하는 데이터만 반환
- 기본적인 JOIN 방식
- 예시

```
SELECT E.NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
INNER JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

INNER JOIN

```
SELECT E.NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
INNER JOIN DEPARTMENTS D  
    ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

```
SELECT E.NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E, DEPARTMENTS D  
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

LEFT OUTER JOIN

- LEFT 테이블의 모든 행과 일치하는 RIGHT 테이블의 데이터를 반환
- 일치하지 않는 경우 NULL 반환
- 예시

```
SELECT E.NAME, D.DEPARTMENT_NAME
```

```
FROM EMPLOYEES E
```

```
LEFT OUTER JOIN DEPARTMENTS D
```

```
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

RIGHT OUTER JOIN

- RIGHT 테이블의 모든 행과 일치하는 LEFT 테이블의 데이터를 반환
- 일치하지 않는 경우 NULL 반환
- 예시

```
SELECT E.NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
RIGHT OUTER JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

FULL OUTER JOIN

- LEFT 테이블과 RIGHT 테이블의 모든 행 반환
- 일치하지 않는 데이터는 NULL로 표시
- 예시

```
SELECT E.NAME, D.DEPARTMENT_NAME
```

```
FROM EMPLOYEES E
```

```
FULL OUTER JOIN DEPARTMENTS D
```

```
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

SELF JOIN

- 같은 테이블을 두 번 사용하여 JOIN 수행
- 주로 계층 구조를 표현할 때 사용
- 예시

```
SELECT E1.NAME AS EMPLOYEE, E2.NAME AS MANAGER  
FROM EMPLOYEES E1  
JOIN EMPLOYEES E2  
ON E1.MANAGER_ID = E2.EMPLOYEE_ID;
```

CROSS JOIN

- 두 테이블 간의 모든 행의 조합을 반환 (Cartesian Product)
- JOIN 조건이 없는 경우 발생
- 결과 집합의 크기가 매우 커질 수 있으므로 조심해서 사용
- 예시

```
SELECT E.NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
CROSS JOIN DEPARTMENTS D;
```

NATURAL JOIN

- 동일한 이름의 컬럼을 기준으로 JOIN
- 조건을 명시하지 않아도 자동으로 연결
- 예시

```
SELECT NAME, DEPARTMENT_NAME  
FROM EMPLOYEES  
NATURAL JOIN DEPARTMENTS;
```

JOIN 사용 시 유의사항

- JOIN 조건 누락

JOIN 조건이 없으면 CROSS JOIN과 동일한 결과 발생

- NULL 처리

OUTER JOIN 시 NULL 값을 처리하는 로직 필요

- 성능

큰 테이블 간 JOIN은 성능 저하 가능, 적절한 인덱스 설정 필요

JOIN 예제

- 고객 이름과 그 고객이 보유한 계좌 유형을 조회

```
SELECT C.NAME, A.ACCOUNT_TYPE
```

```
FROM CUSTOMERS C
```

```
INNER JOIN ACCOUNTS A
```

```
ON C.CUSTOMER_ID = A.CUSTOMER_ID;
```

JOIN 예제

- 고객 이름과 그 고객이 수행한 거래 금액

```
SELECT C.NAME, T.AMOUNT
```

```
FROM CUSTOMERS C
```

```
INNER JOIN ACCOUNTS A
```

```
    ON C.CUSTOMER_ID = A.CUSTOMER_ID
```

```
INNER JOIN TRANSACTIONS T
```

```
    ON A.ACCOUNT_ID = T.ACCOUNT_ID;
```

JOIN 예제

- 모든 고객의 이름과 각 고객이 보유한 계좌 유형을 조회

```
SELECT C.NAME, A.ACCOUNT_TYPE
```

```
FROM CUSTOMERS C
```

```
LEFT OUTER JOIN ACCOUNTS A
```

```
ON C.CUSTOMER_ID = A.CUSTOMER_ID;
```

JOIN 예제

- 모든 고객과 지점을 조합하여 조회

```
SELECT C.NAME AS CUSTOMER_NAME,  
       B.NAME AS BRANCH_NAME  
FROM CUSTOMERS C  
CROSS JOIN BRANCHES B;
```

오라클 JOIN

- 직원 테이블을 기준으로 부서 테이블을 OUTER JOIN

```
SELECT E.EMPLOYEE_ID,  
       E.NAME AS EMPLOYEE_NAME,  
       D.DEPARTMENT_NAME,  
       D.LOCATION  
  
FROM EMPLOYEES E, DEPARTMENTS D  
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID (+)  
      AND D.LOCATION = 'New York';
```

SQL 작성 4

각 지점의 이름과 대출 신청 고객의 정보를 출력하시오. (대출 신청이 없는 지점도 포함)

잔액이 50,000 이상인 계좌를 가진 고객의 이름과 이메일을 출력하시오.

대출 상태가 PENDING인 대출을 신청한 고객 이름과 대출 금액을 조회하시오.

서브쿼리

서브쿼리(Subquery)

- SQL문 안에 포함된 또 다른 SQL문
- 메인 쿼리에 값을 제공하거나 조건을 지정하는 역할
- 일반적으로 괄호로 감싸져 있으며, SELECT, INSERT, UPDATE, DELETE 문에서 사용 가능

서브쿼리 분류

- 연관 여부에 따른 분류: 비연관 서브쿼리, 연관 서브쿼리
- 위치에 따른 분류: 스칼라 서브쿼리, 인라인 뷰, 중첩 서브쿼리
- 반환값 유형에 따른 분류: 단일 행 서브쿼리, 다중행 서브쿼리, 다중열 서브쿼리

연관 여부에 따른 분류

비연관 서브쿼리(Non-correlated Subquery)

- 서브쿼리가 독립적으로 실행되어 결과를 반환
- 메인 쿼리와 서브쿼리 간 상호작용 없음
- 예시: 평균 급여보다 높은 급여를 받는 직원 조회

```
SELECT NAME, SALARY
```

```
FROM EMPLOYEES
```

```
WHERE SALARY > (SELECT AVG(SALARY) FROM EMPLOYEES);
```

연관 서브쿼리(Correlated Subquery)

- 서브쿼리가 메인 쿼리의 각 행마다 실행
- 메인 쿼리의 데이터를 서브쿼리에서 참조
- 예시: 동일한 부서에서 급여가 가장 높은 직원 조회

```
SELECT DEPARTMENT_ID, NAME, SALARY
```

```
FROM EMPLOYEES E1
```

```
WHERE SALARY = ( SELECT MAX(SALARY)
```

```
FROM EMPLOYEES E2
```

```
WHERE E1.DEPARTMENT_ID = E2.DEPARTMENT_ID )
```

```
ORDER BY DEPARTMENT_ID;
```

위치에 따른 분류

스칼라 서브쿼리(Scalar Subquery)

- 서브쿼리가 컬럼 값으로 반환되어 SELECT 절에서 사용
- 주로 집계 값이나 계산 결과를 포함할 때 활용
- 예시: 각 직원의 급여와 해당 부서의 평균 급여를 조회

```
SELECT DEPARTMENT_ID, NAME, SALARY,  
       (SELECT AVG(SALARY) FROM EMPLOYEES  
        WHERE DEPARTMENT_ID = E.DEPARTMENT_ID) AS AVG_SALARY  
FROM EMPLOYEES E  
WHERE DEPARTMENT_ID IS NOT NULL;
```

인라인 뷰(Inline View)

- FROM 절에서 서브쿼리 결과를 테이블처럼 사용
- 별칭을 붙여 활용 가능
- 예시: 부서별 평균 급여를 계산하고 각 부서의 이름과 평균 급여를 조회

```
SELECT D.DEPARTMENT_NAME, T.AVG_SALARY  
FROM DEPARTMENTS D,  
      (SELECT DEPARTMENT_ID, AVG(SALARY) AS AVG_SALARY  
       FROM EMPLOYEES GROUP BY DEPARTMENT_ID) T  
WHERE D.DEPARTMENT_ID = T.DEPARTMENT_ID;
```

중첩 서브쿼리(Nested Subquery)

- WHERE 절에서 조건을 설정하기 위해 서브쿼리 사용
- 서브쿼리 결과에 따라 메인 쿼리의 필터링 결정
- 예시: 특정 부서에 속한 직원 조회

```
SELECT NAME
```

```
FROM EMPLOYEES
```

```
WHERE DEPARTMENT_ID = (SELECT DEPARTMENT_ID
```

```
FROM DEPARTMENTS
```

```
WHERE DEPARTMENT_NAME = 'Department 1');
```

중첩 서브쿼리(Nested Subquery)

- HAVING 절에서 조건을 설정하기 위해 집계 함수와 함께 활용
- 예시: 평균 급여 이상인 부서의 총 급여 조회

```
SELECT DEPARTMENT_ID, SUM(SALARY) AS TOTAL_SALARY
FROM EMPLOYEES
GROUP BY DEPARTMENT_ID
HAVING AVG(SALARY) > (SELECT AVG(SALARY)
                        FROM EMPLOYEES );
```

반환값 유형에 따른 분류

단일 행 서브쿼리(Single-row Subquery)

- 서브쿼리가 단일 행과 단일 열을 반환
- 비교 연산자(=, <, >, 등)와 함께 사용
- 예시: 가장 높은 급여를 받는 직원 조회

```
SELECT NAME
```

```
FROM EMPLOYEES
```

```
WHERE SALARY = (SELECT MAX(SALARY)
```

```
FROM EMPLOYEES);
```

다중 행 서브쿼리(Multi-row Subquery)

- 서브쿼리가 여러 행을 반환
- 다중 행 연산자(IN, ANY, ALL)와 함께 사용
- 예시: 특정 부서들에 속한 직원 조회

```
SELECT NAME
```

```
FROM EMPLOYEES
```

```
WHERE DEPARTMENT_ID IN (SELECT DEPARTMENT_ID
```

```
FROM DEPARTMENTS
```

```
WHERE LOCATION = 'New York');
```

다중 열 서브쿼리(Multi-column Subquery)

- 서브쿼리가 여러 열을 반환
- 메인 쿼리의 여러 열과 비교 가능
- 예시: 특정 부서와 직무를 가진 직원 조회

```
SELECT NAME  
FROM EMPLOYEES  
WHERE (DEPARTMENT_ID, JOB_ID) IN (  
    SELECT DEPARTMENT_ID, JOB_ID  
    FROM JOB_HISTORY );
```

서브쿼리 사용 시 유의사항

- 성능 최적화
 - 비연관 서브쿼리는 독립적으로 실행되므로 더 빠름
 - 연관 서브쿼리는 메인 쿼리의 각 행마다 실행되므로 성능 저하 가능
- 적절한 반환값 확인
 - 서브쿼리가 반환하는 값의 유형(단일 값, 다중 값)에 따라 연산자 선택이 중요
- JOIN 대체 가능성
 - 일부 서브쿼리는 윈도우 함수로 대체할 수 있어 성능 개선 가능

서브쿼리 예제

- 가장 많은 잔액을 가진 계좌의 잔액과 계좌 ID를 조회

```
SELECT ACCOUNT_ID, BALANCE
```

```
FROM ACCOUNTS
```

```
WHERE BALANCE = (SELECT MAX(BALANCE)
```

```
FROM ACCOUNTS);
```

서브쿼리 예제

- 고객 중 가장 최근에 가입한 고객의 이름과 가입 날짜를 조회

```
SELECT NAME, CREATED_AT
```

```
FROM CUSTOMERS
```

```
WHERE CREATED_AT = (SELECT MAX(CREATED_AT)
```

```
FROM CUSTOMERS);
```

서브쿼리 예제

- 잔액이 평균 잔액보다 높은 계좌를 조회

```
SELECT ACCOUNT_ID, BALANCE
```

```
FROM ACCOUNTS
```

```
WHERE BALANCE > (SELECT AVG(BALANCE)
```

```
FROM ACCOUNTS);
```

서브쿼리 예제

- 각 지점에서 대출 금액이 가장 높은 대출을 조회

```
SELECT BRANCH_ID, LOAN_ID, AMOUNT  
FROM LOANS L1  
WHERE AMOUNT = (  
    SELECT MAX(AMOUNT)  
    FROM LOANS L2  
    WHERE L1.BRANCH_ID = L2.BRANCH_ID );
```

서브쿼리 예제

- 각 계좌의 거래 중에서 가장 큰 거래 금액을 조회

```
SELECT ACCOUNT_ID, TRANSACTION_ID, AMOUNT
FROM TRANSACTIONS T1
WHERE AMOUNT = (
    SELECT MAX(AMOUNT)
    FROM TRANSACTIONS T2
    WHERE T1.ACCOUNT_ID = T2.ACCOUNT_ID );
```

SQL 작성 5

지점 위치가 "Location 1"에 해당하는 지점에서 대출을 받은 고객의 이름을 출력하시오.

특정 고객(CUSTOMER_ID=100)이 소유한 모든 계좌에서 발생한 거래를 출력하시오.

각 고객이 보유한 계좌 중 가장 높은 잔액을 가진 계좌를 조회하시오.

집합 연산자

집합연산자

- SQL에서 두 개 이상의 SELECT 결과를 결합하여 하나의 결과 집합을 생성
- 데이터베이스에서 집합 이론을 기반으로 작동
- 서로 다른 쿼리의 결과를 비교하거나 결합할 때 유용

집합연산자 종류

- UNION: 두 SELECT 결과를 합치되 중복 제거
- UNION ALL: 두 SELECT 결과를 합치되 중복 포함
- INTERSECT: 두 SELECT 결과의 교집합
- MINUS: 첫 번째 SELECT 결과에서 두 번째 SELECT 결과를 뺀 차집합

UNION

- 두 SELECT 결과를 합친 뒤 중복된 행은 제거
- 결과는 정렬된 상태로 반환
- 문법:

```
SELECT 컬럼명 FROM 테이블1
```

```
UNION
```

```
SELECT 컬럼명 FROM 테이블2;
```

UNION ALL

- 두 SELECT 결과를 합치며 중복된 행도 포함
- 결과는 정렬되지 않음
- 문법:

```
SELECT 컬럼명 FROM 테이블1
```

```
UNION ALL
```

```
SELECT 컬럼명 FROM 테이블2;
```

INTERSECT

- 두 SELECT 결과의 교집합 반환
- 중복된 행은 하나만 반환
- 문법:

```
SELECT 컬럼명 FROM 테이블1
```

```
INTERSECT
```

```
SELECT 컬럼명 FROM 테이블2;
```

MINUS

- 첫 번째 SELECT 결과에서 두 번째 SELECT 결과를 제외한 차집합 반환
- 중복된 행은 하나만 반환
- 문법:

```
SELECT 컬럼명 FROM 테이블1
```

```
MINUS
```

```
SELECT 컬럼명 FROM 테이블2;
```

집합연산자 사용 시 유의사항

- SELECT 절에 나열된 컬럼 수와 데이터 타입이 동일해야 함
- 결과 집합의 순서는 기본적으로 보장되지 않음
- ORDER BY는 최종 결과에만 적용 가능
- 대량의 데이터 처리 시 성능에 유의해야 함
- ALIAS는 맨 처음 SELECT 절을 따름

집합연산자 예제

- CUSTOMERS와 LOANS에서 공통적으로 존재하는 고객 ID를 조회

```
SELECT CUSTOMER_ID FROM CUSTOMERS
```

```
INTERSECT
```

```
SELECT CUSTOMER_ID FROM LOANS;
```

집합연산자 예제

- TRANSACTIONS에서 거래 유형과 ACCOUNTS의 계좌 유형을 중복 없이 결합하여 조회

```
SELECT TRANSACTION_TYPE FROM TRANSACTIONS
```

```
UNION
```

```
SELECT ACCOUNT_TYPE FROM ACCOUNTS;
```

SQL 작성 6

대출을 받지 않은 고객의 ID를 출력하시오.

거래 테이블과 계좌 테이블에 공통적으로 존재하는 계좌 ID를 출력하시오.

대출 내역이 없는 지점 ID를 조회하시오.

Thanks!

Any questions?

You can find me at:

- ✓ <https://youtube.com/@sql>
- ✓ <https://jungmina.com>
- ✓ Jungmina.com@gmail.com